

Source: ^{Youtube.} The complete Bash scripting Course full - Link: <https://course.jsap.sh> 7:21 h

Basic file

chapter 01
Section 01

↓ ↑
upPanel
down
in history

↳ \$ Pwd: where you are? ↳ LS: show All file in directory
↳ show location ↳ user/dev/test

↳ \$ touch: create file ↳ rm: remove Any file and Delete

↳ mv: move file1 file2 => or Rename ↳ rm -i → delete with Iteration. (Y/N)

↳ alias: → update code and command to what you want => alias rm = rm -i

↳ history: all of history you write in command-Terminal

→ Section 02 : hidden files.

touch .file.txt → LS -a → consee hidden files.
↳ hidden option.

↳ cat file.txt → read file.txt in Terminal

↳ grep: word read from : Search word in Read from files.

=> grep test file.txt ↳ 'dave' → only start with dave.

↳ echo: Print in Terminal

↳ 'dave\$' → End with

↳ -A1 → After line

↳ -B1 → Before line

↳ -c1 → Get context

Command 1 > file.txt → Put command 1 output in file.txt (write data)

↳ >> => append → write end of file.

grep options:

↳ -i → upper x lower case } => -io x -oi

↳ -o → only match word

↳ 2nd Page...

sam

↳ Command 1 | Command 2 → Command 2 recd Command 1 output

→ cat file.txt | grep test | grep test2 → many | and Powerful

→ Paging files:

↳ less: Paging txt file like Page → q to quit. ↑↓jK

↳ / → Command to Search in txt file.

→ Chapter 02

man Pages.

↳ man command ⇒ show document About command ↓↑jK

→ many information

→ 1, --, 10 → command options. → need read ... man man

→ help → for Bash Built in commands. → like history

↳ compgen: show All of thing Built in Bash

↳ type: show location of Built in command ⇒ Kill in bin/Kill

↳ file: say file type → file txt.txt → ASCII text.

"In linux Everything is a file → every command is Bash file
than in bin/ -- " → \$PATH → show PATH of file.

↳ tr: 'n' → Translation → show option.

Start Scripting:

s.a.m

Editor: Idle → VIM → sudo apt install VIM → viming a file

VIM: Terminal Editor in mac/linux → I → for normal mode

↳ Esc → for Exiting mode.

Bash → only linux command with Programming features.

Variable:

Var1 = 'Test' or Var2 = 12

↳ bat → sudo apt install bat → Rust Base.

↳ viwer for text file in Terminal.

~~Permissions~~: Permission: قابلیت اجازت، بیرونی

↳ Chmod

→ + x : executive	} X, r, w
→ + r : read	
→ + w : write	

⇒ #!/bin/cnv Bash

→ Run in Bash

★ first of file

→ Start Scripting:

● ↳ rm : remove file → Dangers.

↳ check Problems.

→ Bash -n → only simple code check.

↳ cp: copy — — — → file1 → file directory

→ Echo \$? → end last output

↳ For in Bash:

```
for thing in ex1 ex2 ex3; do
  --- echo "thing ex is $thing" → 4 Blink
done
```

↳ Any command
↳ end of for

[User Input:]

↳ read → in Bash file. → -P "Prompt to show in Terminal"

→ hello.sh

read -p 'enter name' name

echo 'hi \$name'

Save in name variable.

\$: if you call a variable value.

echo dave | ./hello

→ you can pipe

Argument → on Terminal Input. → name=\$1 → number

→ hello.sh ex1

if: if [[-n ~]]; then

Command1

else

Command2

fi → end of if

→ useful ex:

for thing in "\$@"; do

echo "thing is \$thing"

done

→ Looping All of Arguments.

→ unlimited Argument

→ unlimited Argument

→ unlimited Argument

→ unlimited Argument

→ hello.sh

→ hello.sh one two three

→ functions: → like mini program

function {

Command1

Command2

}

var=\$(function)

→ Conditionals: loop.

while [[-f Command/File]]; do

Command 1

Command 2

Command 3

done

-f → check if file exist

help test -

→ help to file operations

in Linux:

0 → True 1 → False

" " → allow variable

! → dont allow variable.

"\$a is variable" ✓

"\$a is not variable" ✗

s.a.m

→ for-loop:

↳ for thing in {Start .. end}; do

echo "thing is \$thing"

done

num or Anything → start
:
:
:
end

C style:

↳ for ((i=0; i<max; i++)); do

command

done

→ input-output: readinput:]

→ chapter 04

↳ Case Statements:

↳ ~~case~~ Case "\$V" in

one)

echo hi one

;;

end

Command

two)

echo hi two

;;

these)

echo hi these

;;

پایان

esac

↳ end of case → reverse of case esac

↳ the way to know if the case is correct or not is to check the result.

↳ one) Command ;;

two) Command ;;

اولین مقایسه

در اولویت هست

برای عدم اولویت بقی

از & در آخر گذاشته

لب

= & ;

مقایسه برای مقایسه شدن
Value for comparing

مقدار اول مقایسه

دستور

پایان

مقایسه با

مقایسه

پیام سم

پایان

Command

*) → بی نهایت تکرار

پیام سم

پایان

→ Array: List

↳ name of Array = (foo bar baz) → \${arrayname[0]}

echo "\${arrayname[0]}" → foo

↳ array = (
foo
bar
baz
)

"@" → iteration

"*" → All

→ Copy Array → Second Array = ("\${mainArray[@]}")

↳ meant All of items.

↳ declare → option variable name

↳ -i → int ↳ -r : only readable.

↳ -A → Array Associative.

↳ -a → Array

python

↳ works with Key. → like dictionary in python.

↳ -p → inspect variable.

↳ declare -A user ⇒ user = {

user[name] = "test"

user[Age] = 2.

"name": "Rwin",
"Age": 20,

}

→ if's variable: default value for space and Blinks.

↳ apple Banana orange.

↑
IFS

↑
IFS

IFS = " " → Also we can change IFS

→ Command Substitution:

↳ mkdir: make directory

↳ move All file to directory ⇒

directory name

↳ mv file1 file2 fileEx fileend directory/

\$(command) ⇒ give command

{ } ⇒ give variable

↳ in func: thing = \$(my-func)

↳ in func: thing = \${my-func}

copy in func

→ copy in full program and script.

→ Arithmetic Expression:

↳ ((math operation))

بیس، چیک، یا بی

↳ 0 in math means false → and false mean 1

sam

\$: "مقدار متغیر، یا به"

↳ \$? → بیس متغیر دستور، طبق

↳ \$0 → name of script.sh

↳ \$numbers → Argument.

↳ # \$ → Argument num.

↳ \$@ → All of Arguments

↳ \$(command) → Command

Put Command output in var.

→ test = \$(pwd)

\$(name) → Better to Identify

↳ like f string.

vim: %s / text1 / text2

↳ command to Replace text1 and text2 place

→ Process substitution:

↳ creating and write scripts in Bash (A file)

You can put a command in Bash file and it will Run like in Terminal.

↳ Some usefull Commands: not Bash Build in. changing

↳ tr: change Type of something → tr , 'it'

↳ cut -d , -f 1 → cut field
2
3
-num → from end

ex:

echo "I love linux"

→ sed 's/linux/Bash/'

output:

↳ "I love Bash"

↳ sed: for change in txt → line Base.

↳ sed 's/text to find / replacement text'

Substitute ← جایگزینی جایگزینی
-i ⇒ write in file.

حاشه با و یا " ⇒ متن اول

↳ awk: awk '{Print \$1}' file.txt ⇒ Powerful linux tool used for Processing and Analyzing text → ~~str~~ structured data and columns

↳ find: find Location 1/Location n -type { f : file

↳ finding Anything At everywhere. { d : directory

{ l : other type

Search
filter
extract
calculate
format

→ Bash Arguments.

↳ Bash -x ⇒ Run in Debug mode.

↳ Print everything in Terminal.

⇒ set: you can Add Argument in Bash file.

PS4: [list thing debug mode print]

⇒ set -x

⇒ PS4="[debug]" Bash -x simble Bash.sh

→ does not Run code.

↳ [debug] Command out

⇒ Bash -n: check syntax

[debug] Command out

⇒ Bash -u: Check undefined and raise error.

[debug] Command out

→ **Shellcheck**: Powerful checker for Bash files.

↳ shellcheck file.sh

check source Actually works:

• .lib | greeting || exit 1

Location of file

→ **Source coding**: creating lib. and modules.

↳ **Source** Location of Lib (Bash file) → Source a file.

>Bash:

```
if ! (return 2) >/dev/null; then
```

Command
function

```
fi
```

→ we are being called directly

→ check The Real Source file is Running

>Python:

```
if "__name__" == "__main__":
```

```
    main()
```

Command

function()

→ name = main → main file Running

→ **Return VS output**:

→ **Parameter Expansion**:

↳ "\$#{var}" → len of var ↳ "\${var^^}" → only D. upper

↳ "\${var^}" → first letter large ↳ "\${var,,}" → lower case

↳ "\${var^^}" → All uppercase

↳ \$var = \${2:- example} → Default value if not given.

↳ Var = \${1:?} → Argument is Requirement.

↳ can write Err message.

↳ \${1:?} 'You must give var'

↳ echo "\$PATH/a/a" → replace a → 0

↳ for iterate

Python))

↳ echo "\${var:1:2:3}" → Start character → like List [0:1]

↳ 1,2,3 → Backwards.

↳ \${var:1:-1}

Start point

↳ end point

S='dave Eddy'

len=\${#S}

for ((i=0; i<len; i++));do

c=\${S:i:1}

echo \$c

done

→ more custom than echo

dave eddy

↳ **Printf**: Print formatted text and values to the Terminal.

→ `Printf "name: %s\nAge: %d\n" "Ali" 2.`

→ `Printf %s\n "${arr[@]#b}"` → delete B

→ **Basic Globbing: / Extended Globbing:**

→ **shopt**: Controls optional features and shell behavior in Bash

→ You can read more.

→ **Brace Expansion**: قابلیت روی دادن به لیست

→ `arr = ( "${filename}. {txt,jpg,mov} )` → `filename.txt | ".jpg | .mov`

→ example: `>> touch {foo,bar,baz}. {jpg,txt}` ⇒ # 6 out put

→ **numeric Brace Expansion**:

→ `echo {1..100..5}` → 1 to 100 with 5 steps.

→ 1, 6, 11, ... , % {1..9..3}

→ **Seq** 1 10 → End → like

Start

→ 3 to 2 = 6

→ `Printf %10s %10s\n`

→ `var var2`
S → mean variable.

→ **Date formatting**:

تغییر نحوه نمایش تاریخ و ساعت
خواب این پیشتر میبود شبی با دستبر
man/help

→ **Date**: shows date of now: Wed Nov 26 23:50:55 Est 2025

→ **Datefmt**: 'day month day time zone Year' → mon ~~the~~ date:

→ **Seconds**: number of seconds that shell has been running.

→ and more: ⇒ # 102748

You can read more
About date.

→ **Regular Expression: Regex**: Searching text in files.

→ `grep "hi" logfile.txt`

مطابقت با الگوریتم
Test whether matches

Symbol	<code>^</code>	<code>\$</code>	<code>.</code>	<code>*</code>	<code>+</code>	<code>?</code>	<code>[abc]</code>	<code>[0-9]</code>	<code>[^0-9]</code>
meaning	Start	end	Any ch	Zero more	one more	Zero one	یکی abc	Any 0-9	Any ch except digit
ex	<code>^root</code>	<code>root\$</code>	<code>C.t</code>	<code>ab*</code>	<code>ab+</code>	<code>colou.r?</code>	<code>[arr]</code>	<code>[0-9]</code>	<code>[^0-9]</code>

→ **map files/mapfile**: Reads lines from Input and stores them in Bash Array.

→ **mapfile users <users.txt**

users:

- dave
- arvin
- github

→ **[[vs Test**: Bash brackets:

↳ different bracket types are used for conditions, arrays, commands and...

[], **test** → for simple works.

[[]] → **Advance Test**: → also can use Regex.

↳ Code >>

```
if [[ $name =~ ^A ]]; then
    echo "Starts with A"
fi
```

(()) =

↳ Arithmetic Evaluation
برای حساب جبری، ریاضی و غیره

↳ **-gt**: greater than: → for num/Int

↳ **[[10 > 9]]**; → false → 1

↳ **[[10 -gt 9]]**; → True → 0

check String Base

[] → Condition

[[]] → Advance

() → Subshell

\$() → get command out put

(()) → math

{ } → Group command

\${ } → work w variable

→ **Trap signals**:

↳ **Trap -l** → can see every signal

↳ executes a command when shell receives a signal or exits.

Sigint → **Ctrl-C**

↳ **Trap 'echo "Test Err"' Sigint** → show Test Err when press **Ctrl-C**

ctrl-Z
for Kill

sam

→ **named pipes**: Special files that Allow Process to communicate through a shared data stream.

به کمک آنها، ارتباط بین دستورات، لینک می شود

→ **mkfifo myPipe**: create pipe ⇒ `ls -l myPipe`

→

→ wait for myPipe
`cat <myPipe`

`echo "Hello from another Process" > myPipe`

end of Command &

→ Run command in Background

→ job : see running

Kill jobid.

end running command

→ **exec command**: Runs a command or changes file descriptions. → shell به ترتیب به اجرا گذاردن دستورات

→ `exec 3 >file.txt`

به جای فایل، به اجرای دستورات

→ `echo "hi" >&3`

<> file name

→ To read and write file

`exec {fd}<>pipe`

به جای فایل، به اجرای دستورات

{fd} به جای دستورات

→ **Color output**: Uses Ansi escape to display colored text in Terminal.

→ `echo | printf → '\e[1m text \n'` → **Bold text** } → you can Also change Background color...

Color codes
text:

`\e[0m` → reset

→ "31 → red
→ "32 → green
→ "33 → yellow
→ "34 → Blue

→ 35 → magenta
→ 36 → cyan
→ 37 → white
→ 38 → Reset

→ you can read move about it.

Bash color

Codes

`for i in {0..255}; do`

`printf '\e[38;5;dm Color %d \e[0m\n' $i`

`done`

→ shows all 256 color formats.

Cursor Commands: Control the Position and visibility of the Terminal

Cursor.

fork bomb: $\hookrightarrow$ مشخص کردن اینکه متن دیتا لحاظ بشود.

[learn more and more] [باید بیشتر بخواند و یادگیری]

PS1 Variable:

$\hookrightarrow$ Defines the Primary Prompt displayed by Bash shell

$\hookrightarrow$ تعریف کردن همدست در پیرو در ترمینال نمایش داده شود.

$\hookrightarrow$ PS1 = "linux >" $\rightarrow$ shell : linux >

! Dont Try

Cursor commands codes

creating func and more with Alias:

$\hookrightarrow$ alias greet = 'echo hello' $\hookrightarrow$ alias() { echo hi \$1; }

Anything

Acknowledgment:

$\hookrightarrow$ Special thanks to Dave Eddy and the you suck at Programming channell for creating and sharing the Bash scripling Course that inspired these notes.

$\hookrightarrow$ These notes are entirely based on my own understanding Interpretation, and writing, if these are any mistake or Inaccuracies, Please forgive them and feel free to correct them, thank you for sharing your Knowledge and making this Course Available to others. $\rightarrow$ Rwin-X

$\hookrightarrow$ 1E[H] $\rightarrow$ move to top-left

$\hookrightarrow$ 1E[2J] $\rightarrow$ Clear screen

$\hookrightarrow$ 1E[K] $\rightarrow$ Clear from cursor to End

$\hookrightarrow$ 1E[5A] $\rightarrow$ move 5 line up

$\hookrightarrow$ 1E[3B] $\rightarrow$ move 3 line Down

$\hookrightarrow$ 1E[4C] $\rightarrow$ move 4 columns Right

$\hookrightarrow$ 1E[2D] $\rightarrow$ move 2 columns left

Progress bar:

for i in {1..5};do

Printf "In Progress : %d/%x" $\$((i*2.))$

sleep 1

done

Printf "\n"

Alt-f: forward word

Ctrl-b:

Back char

Ctrl-E:

swap char

Ctrl a:

get to beginning

Alt-b: Back word

Ctrl-w:

delete one word

Ctrl-f:

forward char

Ctrl E:

get to the End

Alt-t: swap words

Ctrl u:

delete everything